Polynomial Regression

+ Bias-Variance Trade-off

Definitions • equations • examples • diagrams • Colab code

Polynomial | Bias | Variance | Code

Data Science Journey • Part 7 →

Polynomial Regression: What?

Fitting CURVES with a linear model

A regression technique that adds powers of the input (x^2, x^3 …) as new features so a linear model can fit a curved relationship.

Why?

a straight line cannot follow a curve: it underfits

Example

car horsepower vs mpg: mpg drops fast, then flattens

Key idea

still LINEAR in the coefficients, so ordinary least squares still works

The Equation

Degree d polynomial

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d + \varepsilon$$

Degree 1

y = β0 + β1·x straight line

Degree 2

y = β0 + β1·x + β2·x² parabola (one bend)

Degree 3

... + β3·x³ S-shape (two bends)

Solved by least squares (same as linear regression):

$\hat{\beta} = (X^T X)^{-1} X^T y$ where X = [1, x, x², ..., x^d]

How It Works: Feature Expansion

Create new columns, then fit a linear model

1 feature, degree 3

$x = 2 \quad \rightarrow \quad [\,1,\ x,\ x^2,\ x^3\,] = [\,1,\ 2,\ 4,\ 8\,]$

2 features (a, b), degree 2

$[\,1,\ a,\ b,\ a^2,\ a{\cdot}b,\ b^2\,]$ (includes interaction $a{\cdot}b$)

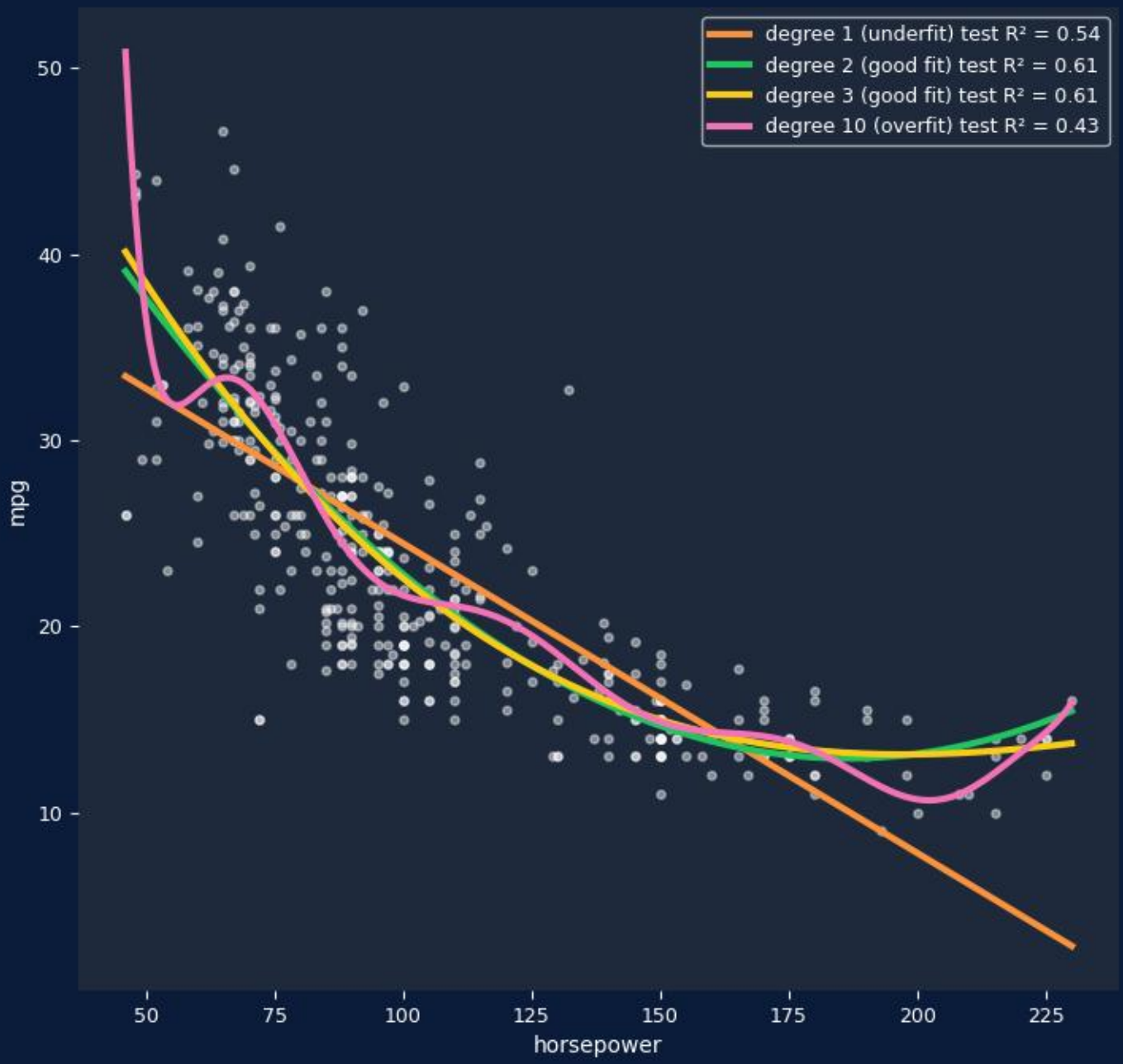
Number of features grows FAST: $C(p + d, d)$

2 original features, degree 2	**6 features**
10 original features, degree 3	**286 features**
100 original features, degree 3	**176,851 features**

This is why high degrees overfit and why scaling + regularisation matter.

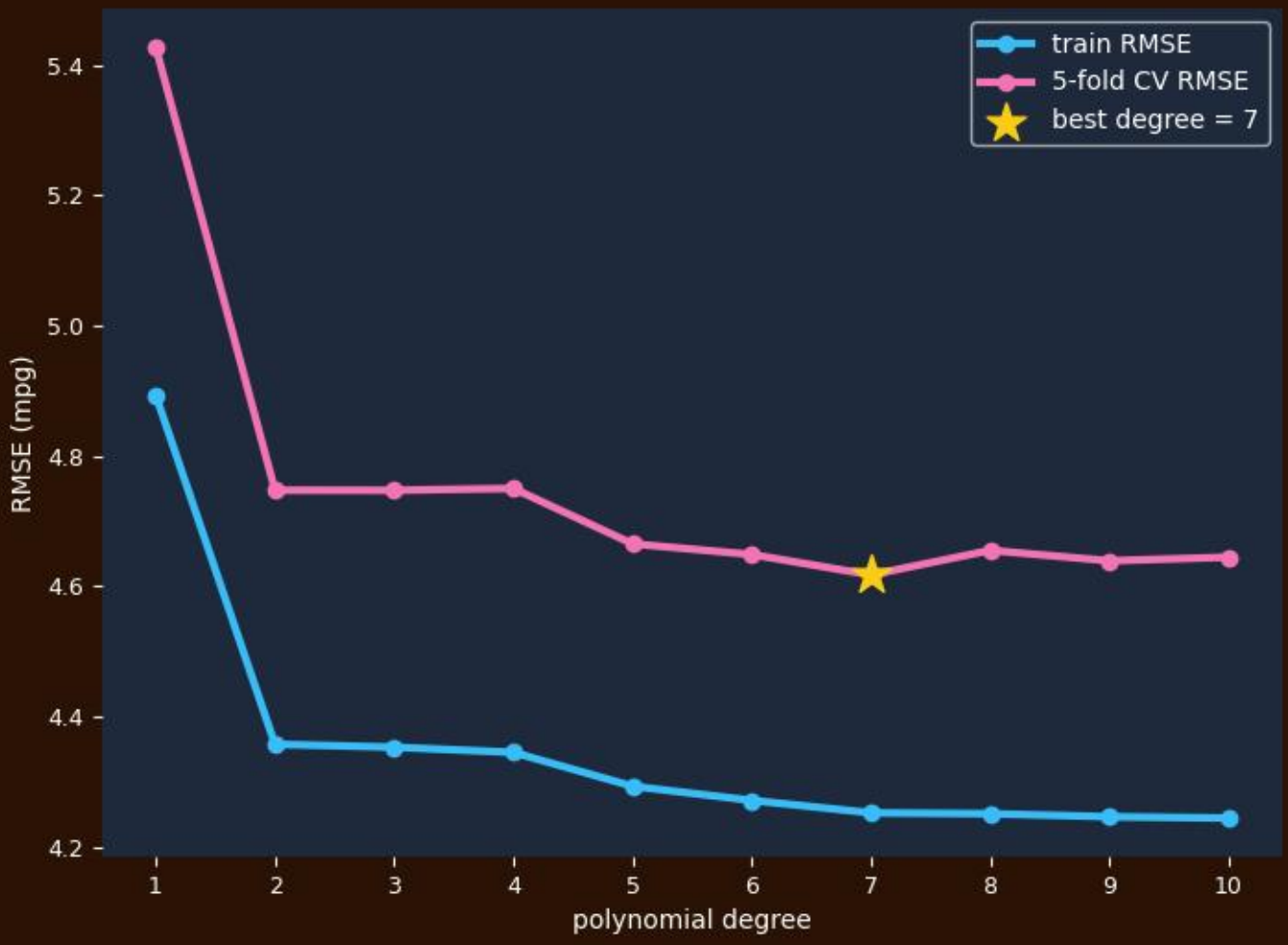
Example: Horsepower vs MPG

392 cars (Auto MPG dataset), fits of degree 1, 2, 3 and 10



Choosing the Degree

Train error keeps falling; CV error flattens after degree 2



CV RMSE: degree 1 = 5.43, degree 2 = 4.75, degree 7 (best) = 4.62.
Degree 2 is almost as good and far simpler: prefer the simpler model.
Pick the degree from validation data, never from training error.

Bias-Variance: What?

Why models make errors

Bias

error from WRONG ASSUMPTIONS (model too simple).
High bias = underfitting. A line forced onto a curve.

Variance

error from SENSITIVITY to the training sample (too complex).
High variance = overfitting. Memorises noise.

Irreducible noise (σ^2)

randomness in the data that no model can remove

Trade-off: lowering one usually raises the other

The Equation + Dartboards

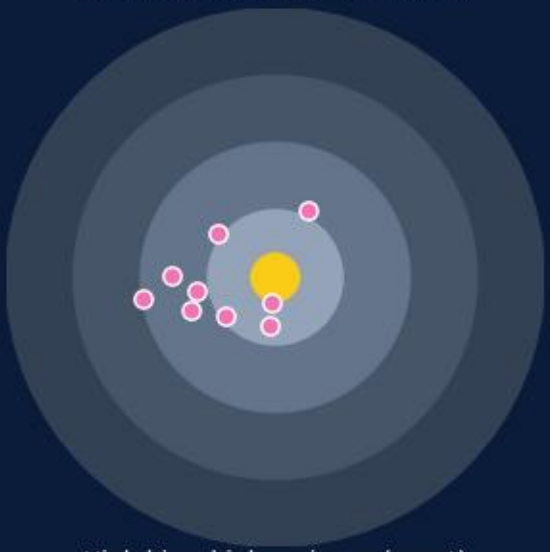
$$E[(y - \hat{f})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

$\text{Bias} = E[\hat{f}] - f$ $\quad$ $\text{Variance} = E[(\hat{f} - E[\hat{f}])^2]$

Low bias, low variance (ideal)



Low bias, HIGH variance (overfit)



HIGH bias, low variance (underfit)

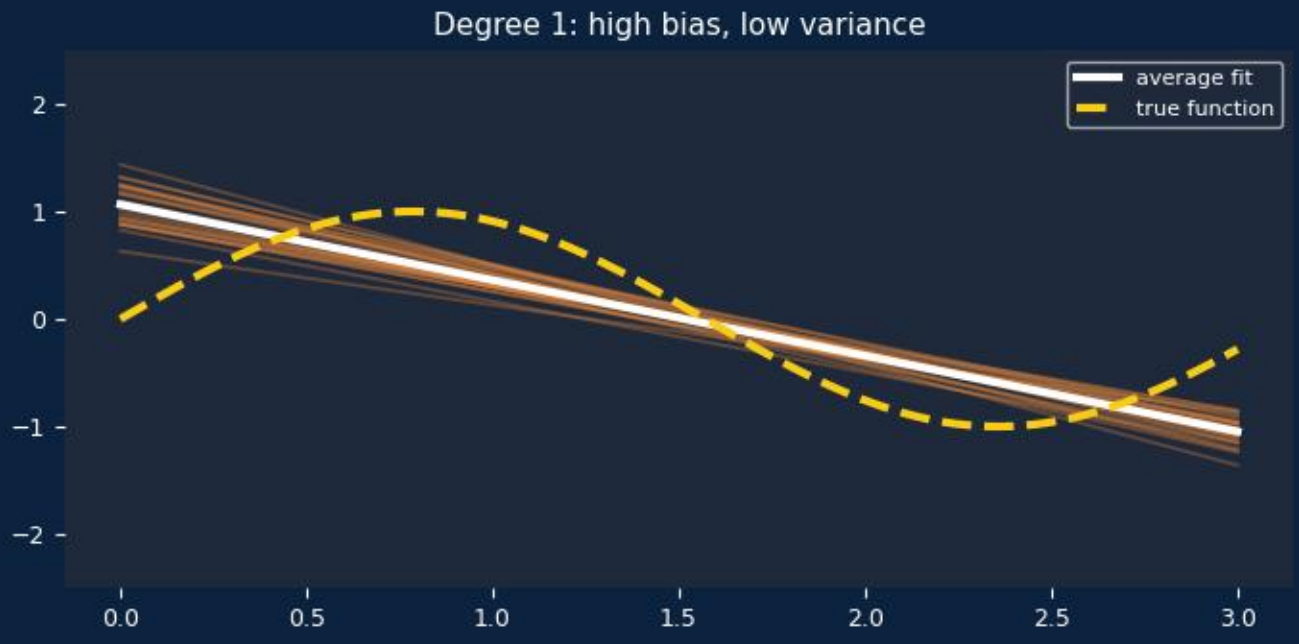


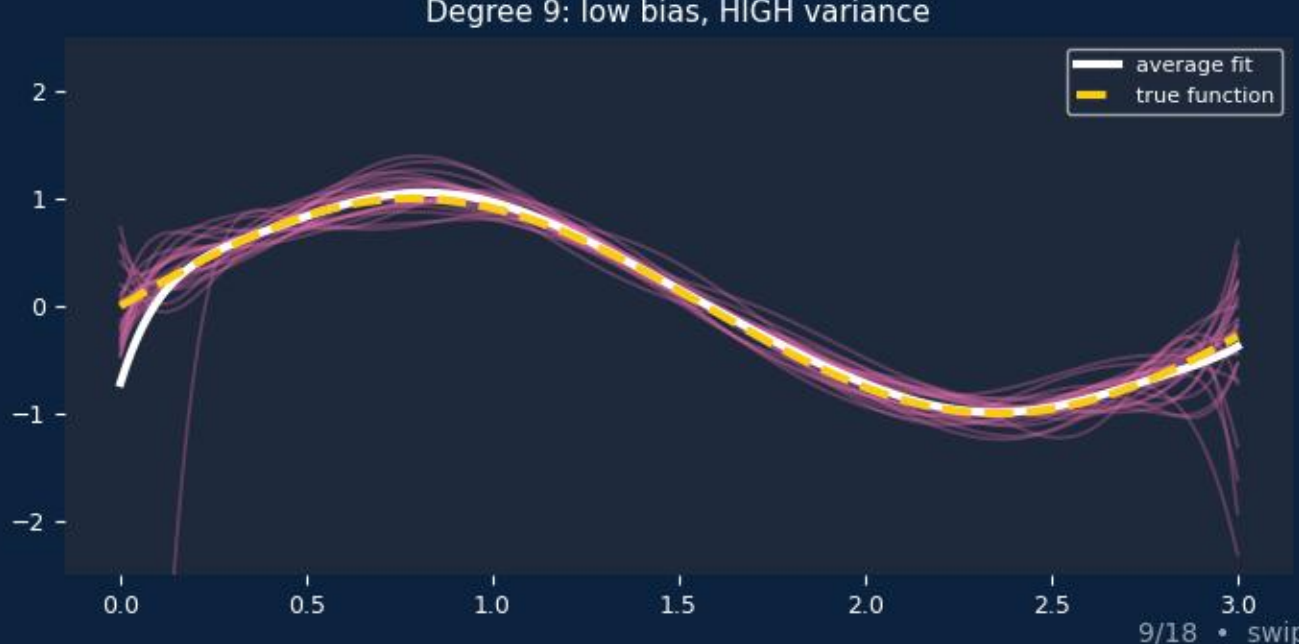
High bias, high variance (worst)



Variance You Can SEE

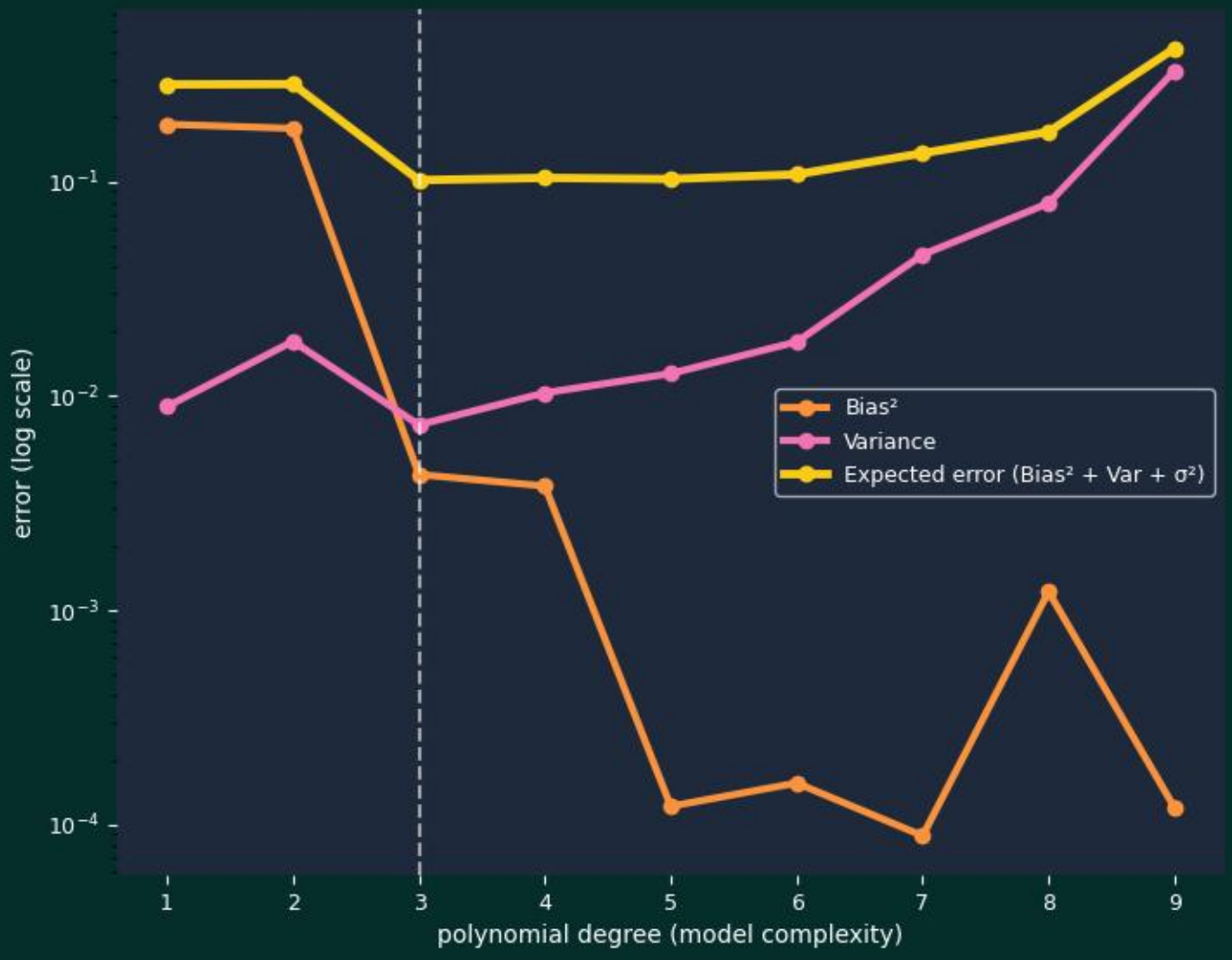
25 different training sets (n = 60), one fitted curve each





Bias² + Variance = Error

Simulation: true f(x) = sin(2x), noise σ = 0.3, n = 60, 200 repeats



Sweet spot: degree 3. Bias² falls as complexity rises, variance rises.
Degree 1 bias² = 0.19; degree 9 variance = 0.33.

Underfit vs Good vs Overfit

Numbers from the simulation (n = 60)

	Underfit	Good fit	Overfit
Degree	1 (too low)	3 (just right)	9 (too high)
Bias²	0.185 HIGH	0.004	0.000
Variance	0.009	0.007	0.328 HIGH
Expected error	0.284	0.102 (lowest)	0.418
Symptom	train and val errors both high	both low and close	train low, validation high

Expected error = Bias² + Variance + σ² (σ² = 0.09 here).

How to Fix Each Problem

High bias (underfitting)

increase the polynomial degree • add features / interactions
reduce regularisation • try a more flexible model

High variance (overfitting)

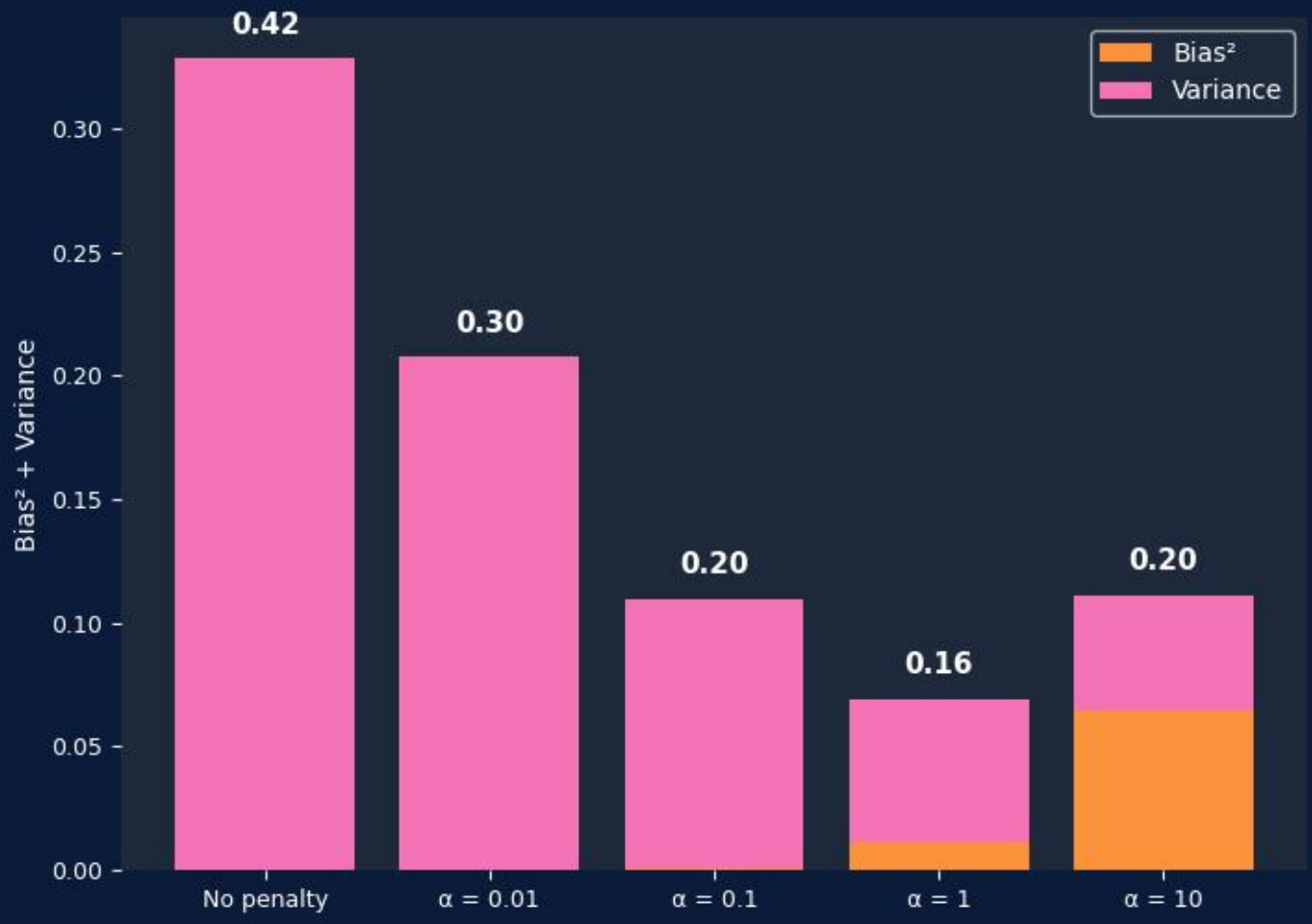
get more training data • lower the degree • regularise (Ridge/Lasso)
use cross-validation • bagging / ensembles • early stopping

Rule of thumb

Plot train vs validation error. Both high → fix BIAS.
Train low, validation high → fix VARIANCE.

Regularisation Tames Variance

Degree 9 polynomial, Ridge penalty α (simulation, n = 60)



Best penalty: α = 1 (error 0.16 vs 0.42 with none).
Too large a penalty adds bias again. Always scale features first.

Code 1: Polynomial Regression

Google Colab • Auto MPG dataset

```
import numpy as np, pandas as pd, seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

df = sns.load_dataset("mpg").dropna()       # 392 cars
X, y = df[["horsepower"]], df["mpg"]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, random_state=42)

# What does PolynomialFeatures do?
print(PolynomialFeatures(3).fit_transform([[2]]))
# -> [[1. 2. 4. 8.]] = [1, x, x^2, x^3]

def poly(d, model=None):                    # scale -> expand -> fit
    return make_pipeline(StandardScaler(),
        PolynomialFeatures(d, include_bias=False),
        model or LinearRegression())

g = pd.DataFrame({"horsepower": np.linspace(46, 230, 200)})
for d in [1, 2, 3, 10]:
    m = poly(d).fit(Xtr, ytr)
    print("degree", d, "test R2:",
          round(r2_score(yte, m.predict(Xte)), 3))
    plt.plot(g, m.predict(g), label=f"degree {d}")
plt.scatter(X, y, s=10, c="gray"); plt.legend(); plt.show()
```

Code 2: Pick the Degree

Google Colab • cross-validation

```
# Choose the degree with cross-validation
from sklearn.model_selection import cross_val_score

rows = []
for d in range(1, 11):
    train = mean_squared_error(y, poly(d).fit(X, y)
                               .predict(X)) ** 0.5
    cv = -cross_val_score(poly(d), X, y, cv=5,
        scoring="neg_root_mean_squared_error").mean()
    rows.append((d, train, cv))

res = pd.DataFrame(rows, columns=["degree", "train_rmse",
                                  "cv_rmse"])
print(res.round(2))
best = int(res.loc[res.cv_rmse.idxmin(), "degree"])
print("Best degree by CV:", best)

plt.plot(res.degree, res.train_rmse, label="train")
plt.plot(res.degree, res.cv_rmse, label="cross-val")
plt.xlabel("degree"); plt.ylabel("RMSE"); plt.legend()
```

Code 3: Bias-Variance + Ridge

Google Colab • simulation and regularisation

```
# Bias-variance by simulation (true function is known)
true_f = lambda x: np.sin(2 * x)
x0 = np.linspace(0, 3, 50).reshape(-1, 1)   # test points
sigma, n = 0.3, 60                          # noise, sample size

def bias_var(d, model=None, S=200, seed=0):
    rng = np.random.default_rng(seed)      # same data for every model
    P = np.empty((S, len(x0)))
    for s in range(S):       # many different training sets
        xs = rng.uniform(0, 3, n).reshape(-1, 1)
        ys = true_f(xs).ravel() + rng.normal(0, sigma, n)
        P[s] = poly(d, model).fit(xs, ys).predict(x0)
    bias2 = ((P.mean(0) - true_f(x0).ravel()) ** 2).mean()
    return bias2, P.var(0).mean()

out = []
for d in range(1, 10):
    b, v = bias_var(d)
    out.append((d, b, v, b + v + sigma ** 2))
bv = pd.DataFrame(out, columns=["degree", "bias2",
                                "variance", "total_error"])
print(bv.round(3))

# Regularisation tames variance (degree 9)
from sklearn.linear_model import Ridge
reg = []
for a in [0, 0.01, 0.1, 1, 10]:
    mod = Ridge(alpha=a) if a else None
    b, v = bias_var(9, mod)
    reg.append((a, b, v, b + v + sigma ** 2))
reg = pd.DataFrame(reg, columns=["alpha", "bias2",
                                 "variance", "total_error"])
print(reg.round(3))
```

Recap

Polynomial regression

add x^2, x^3 … as features; linear in β, curved in x

Equation

$y = \beta 0 + \beta 1x + \beta 2x^2 + \ldots + \beta dx\text{^}d + \varepsilon$

Bias-variance

Error = Bias^2 + Variance + σ^2

Pick complexity

validate it; regularise to cut variance

Underfit or overfit?
What do YOU see more?

Comment your answer

Repost to help a learner

Follow: next is Classification

#DataScience #PolynomialRegression #BiasVariance #MachineLearning #Python